21CSS303T/DATA SCIENCE REPORT

ENERGY INSIGHTS

Submitted by

SAKTHI S – RA2311026050036

P. DURGA PRASAD – RA2311026050047

Under the guidance of

Dr. C GUNASUNDARI

(Associate Professor, School of Computing)

In partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING WITH SPECIALIZATION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, TIRUCHIRAPPALLI

MAY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University U/S3ofUGCAct,1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **ENERGY INSIGHTS** Is the Bonafide work of **SAKTHI S [RA2311026050036], P. DURGA PRASAD [RA2311026050047]**who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Dr. C Gunasundari M.E., Ph.D.,

Associate Professor
School of Computing,
SRM Institute of Science and Technology,
Tiruchirappalli.

SIGNATURE

Dr. R. Balaji Ganesh, M. Tech., Ph.D., Postdoc

Head of the Department
Department of Computer Science and Technology with Specialization in Artificial Intelligence and Machine Learning,
School of Computing
SRM Institute of Science and Technology,
Tiruchirappalli.

Submitted for the project viva-voce held on [Date] at SRM Institute of Science and Technology, Tiruchirappalli.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY TIRUCHIRAPPALLI

DECLARATION

We hereby declare that the entire work contained in this project report titled

ENERGY INSIGHTS has been carried out by

SAKTHI S and **P. DURGA PRASAD** at SRM Institute **of Science and Technology, Tiruchirappalli, under the guidance of Dr. C GUNASUNDARI, Associate Professor, School of Computing.**

Place: Tiruchirappalli

Date: [Date]

ABSTRACT

Energy consumption data plays a crucial role in understanding usage patterns and improving efficiency in modern systems. Analyzing energy usage helps in applications such as smart homes, industrial monitoring, cost optimization, and sustainable resource management. This project presents a data science–based energy analysis system capable of processing and visualizing energy consumption patterns from structured datasets. The system utilizes energy consumption datasets, which contain historical usage records for analysis and evaluation. Data processing techniques including cleaning, transformation, and exploratory data analysis are used to extract meaningful insights from the data. Analytical and predictive models are applied to identify trends and forecast future energy consumption effectively. The system is deployed using a backend framework, enabling efficient data handling and processing through APIs. A user-friendly frontend is developed, providing features such as data upload, graphical visualization, trend analysis, prediction outputs, and an interactive dashboard with charts and history tracking. The system demonstrates reliable performance in identifying consumption patterns and generating useful predictions, highlighting the effectiveness of data science techniques in energy analysis. This project emphasizes the integration of data analytics, backend systems, and modern web technologies to build a complete end-to-end intelligent application.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
1	**INTRODUCTION**	**8**
	1.1 Introduction	8
	1.2 Problem Statement	**Error! Bookmark not defined.**
	1.3 Objectives	9
	1.4 Scope and Motivation	10
2	**EXISTING SYSTEM**	**11**
3	**PROPOSED METHODOLOGY**	**14**
4	**IMPLEMENTATION**	**16**
5	**RESULT AND DISCUSSION**	**22**
6	**CONCLUSION**	**26**
	REFERENCES	27

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.1	System architecture	14
4.7.1	main.py	21
4.7.2	Continuation of main.py	21
4.7.3	AudioUploade.js	22
4.7.4	WaveForm.js	22
4.7.5	EmotionChart.js	23
4.7.6	Api.js	23
5.1.1	Main Emotion AI Dashboard	24
5.1.2	Waveform Dashboard	24
5.1.3	Loading model and labels into the dashboard	25

5.1.4	**Main output Dashboard**	**25**
5.1.5	**Result (emotion) of an audio file**	**26**

LIST OF ABBREVIATIONS

S. No.	Abbreviations	Expansions
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	SCM	Supply Chain Management
4	EDA	Exploratory Data Analysis
5	API	Application Programming Interface
6	DB	Database
7	ERP	Enterprise Resource Planning
8	KPI	Key Performance Indicator
9	UI	User Interface
10	ETA	Estimated Time of Arrival
11	R^2	Coefficient of Determination
12	CSV	Comma-Separated Values
13	GPS	Global Positioning System
14	ETL	Extract, Transform, Load
15	XAI	Explainable Artificial Intelligence

1. INTRODUCTION

1.1 Introduction

Energy consumption is one of the most important aspects of modern life, influencing economic growth, industrial development, and environmental sustainability. With the increasing demand for electricity and energy resources, it has become essential to monitor and analyze energy usage effectively. Energy data contains valuable information that reflects consumption patterns, user behavior, and system efficiency.

In recent years, there has been a growing need for intelligent systems that can analyze large volumes of energy data and provide meaningful insights. Applications such as smart homes, industrial monitoring systems, and energy management platforms require efficient tools to understand usage trends and optimize consumption. Traditional methods of analyzing energy data are often manual, time-consuming, and lack proper visualization, making it difficult for users to interpret the results.

With advancements in data science and machine learning, it is now possible to automatically process energy datasets, identify patterns, and generate predictive insights. This project focuses on developing a data-driven system that analyzes energy consumption data and presents the results through an interactive web-based dashboard. The system aims to help users understand their energy usage, identify inefficiencies, and make informed decisions for better energy management.

PROBLEM STATEMENT

Energy consumption analysis is a challenging task because usage patterns vary significantly across different users, locations, and time periods. Factors such as lifestyle, seasonal changes, and industrial activities influence energy

consumption, making it difficult to accurately understand and interpret usage behavior.

Another problem is the presence of incomplete, inconsistent, or noisy data in energy datasets, which can affect the quality of analysis. Missing values, irregular readings, and data collection errors make it difficult to extract meaningful insights and may reduce the reliability of the system.

In addition, some consumption patterns may appear similar even though they represent different usage behaviors. For example, peak usage during different time periods may have similar values but different underlying causes. This can lead to confusion during analysis and interpretation.

Existing systems are often limited in their ability to handle large datasets efficiently and do not provide proper visualization or user-friendly interfaces. Therefore, there is a need for a system that can automatically process energy data, extract important patterns, and provide accurate insights and predictions using advanced data science techniques.

.

1.3 Objectives

The main objective of this project is to develop a system that can recognize human emotions from speech using deep learning techniques.

The specific objectives of this project are:

- To collect and use an appropriate dataset for speech emotion recognition
- To extract important audio features such as MFCC, chroma, and mel spectrogram
- To design and train a deep learning model for classifying emotions
- To develop a backend system for processing audio and generating predictions
- To create a user-friendly frontend interface for uploading audio and displaying results
- To visualize the predicted emotions using charts, waveform, and history.

1.4 Scope and Motivation

Scope

The main objective of this project is to develop a system that can analyze and visualize energy consumption data using data science techniques.

The specific objectives of this project are:
• To collect and use an appropriate dataset for energy consumption analysis
• To preprocess and clean the data for accurate analysis
• To perform exploratory data analysis to identify patterns and trends
• To design and implement models for analyzing and predicting energy usage
• To develop a backend system for processing data and generating insights
• To create a user-friendly frontend interface for uploading data and displaying results
• To visualize energy consumption patterns using charts, graphs, and dashboards

Motivation

The motivation behind this project is to develop an intelligent system that can analyze energy consumption and provide meaningful insights using data science techniques. Efficient energy usage is essential for reducing costs and promoting sustainability.

With the increasing demand for electricity and the rise of smart technologies, there is a need for systems that can monitor and optimize energy usage effectively. This project aims to explore how data analysis and predictive modeling can be used to understand consumption patterns and improve decision-making.

It also provides an opportunity to integrate data science, backend development, and frontend technologies to build a complete real-world application that addresses practical energy management challenges.

2. EXISTING SYSTEM

In the existing system, energy consumption analysis is mainly performed using traditional methods and basic data analysis techniques. These systems depend on manual calculations and simple tools such as spreadsheets and basic statistical methods to analyze energy usage data.

In many cases, energy data is analyzed using predefined rules or static reports. These approaches require significant human effort and are not efficient for handling large and continuously growing datasets. They also fail to capture complex patterns and hidden trends present in energy consumption data.

Some systems rely only on limited parameters such as total consumption or average usage, which are not sufficient to fully understand energy behavior over time. As a result, these methods often produce less accurate insights when dealing with real-world and dynamic energy data.

Additionally, most existing systems do not provide an interactive interface for users. They lack advanced visualization features such as dynamic charts, dashboards, or historical tracking, making it difficult for users to interpret and analyze the results effectively.

Overall, traditional energy analysis systems are limited in terms of accuracy, scalability, and usability, which creates a need for more advanced, data-driven, and intelligent solutions.

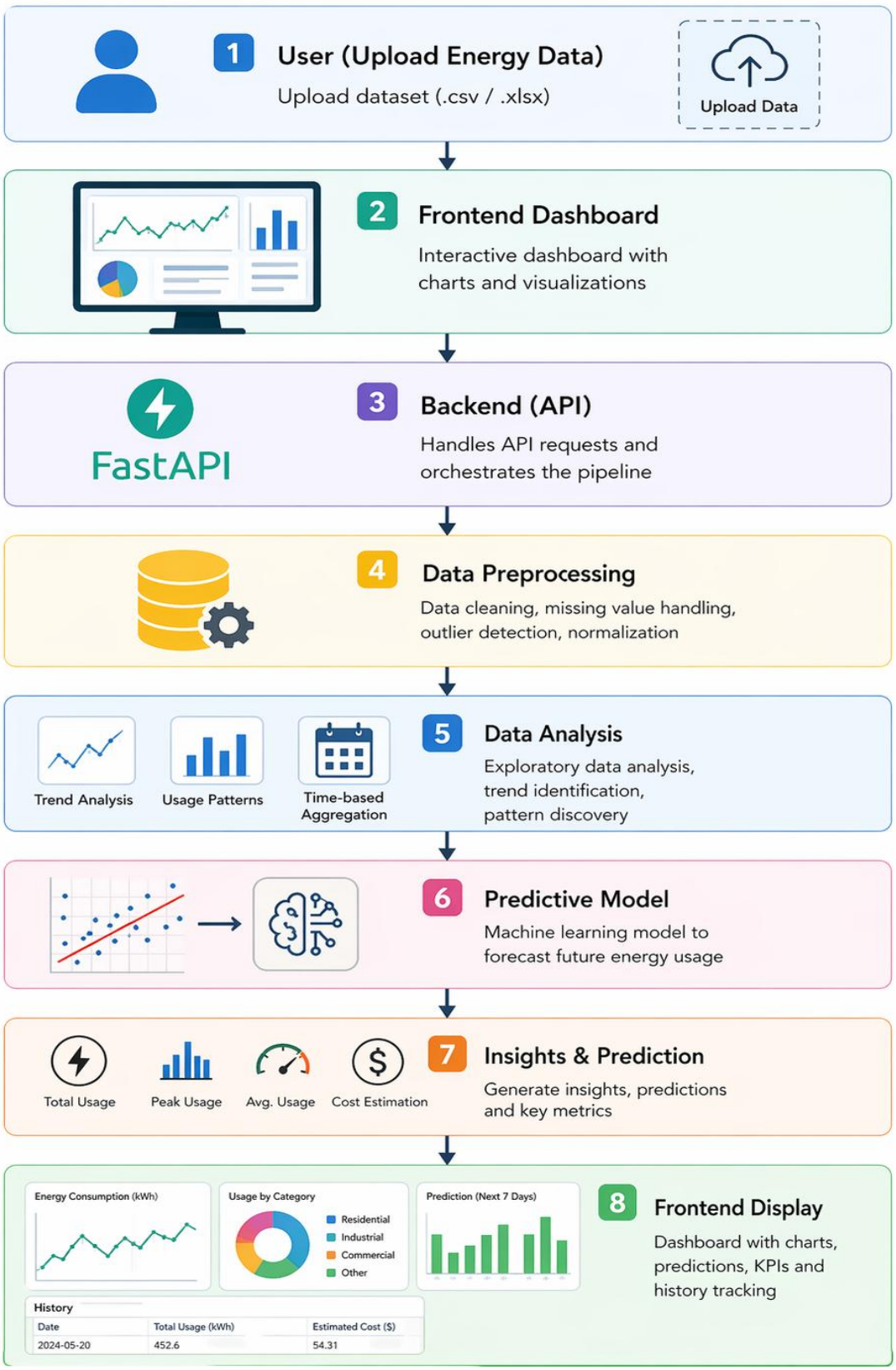


Figure 2.1. SYSTEM ARCHITECTURE

3. PROPOSED METHODOLOGY

The proposed methodology of this project is designed in a clear and step-by-step manner to ensure proper understanding of how the system works from data input to energy insight generation. The system follows a structured pipeline that integrates data science, analytical models, and web-based technologies.

Step1:DataCollectionandStorage
The process begins with collecting an energy consumption dataset. The dataset contains historical energy usage records such as daily or monthly consumption values. These data files (CSV/Excel) act as the primary source for analysis and prediction.

Step2:DataPreprocessing
The collected data is processed to improve quality and consistency. Preprocessing steps include handling missing values, removing duplicates, and normalizing the data. This step ensures that the dataset is clean and suitable for further analysis.

Step3:ExploratoryDataAnalysis
Important patterns and trends are identified using exploratory data analysis techniques. Statistical summaries and visualizations are used to understand energy usage behavior over time. This step helps in identifying peak usage periods, seasonal variations, and anomalies.

Step4:DataAnalysisandFeatureEngineering
Relevant features are selected and transformed to improve analysis and prediction. Aggregations such as daily, weekly, or monthly summaries are created. These features represent key characteristics of energy consumption and are used as input for predictive models.

Step5:PredictiveModel
A machine learning model such as Linear Regression or other suitable algorithms is used to predict future energy consumption. The model is trained using historical data, and the dataset is divided into training and testing sets to ensure accuracy and reliability.

Step6:BackendProcessing
The analytical model is integrated into a backend system. The backend handles data uploads, processes the input dataset, performs analysis, and generates predictions along with key insights.

Step7:UserInteraction

The system allows users to upload datasets and interact with the application through a simple interface. Users can view insights, analyze trends, and understand energy consumption patterns through visualizations.

Overall

The methodology ensures a smooth flow from data input to insight generation, combining data analysis and modern web technologies to create an efficient and user-friendly Energy Insights system.

4. IMPLEMENTATION

The implementation of the proposed system is carried out using a combination of programming, data analysis techniques, backend development, and frontend visualization tools. The entire system is developed in a modular manner to ensure clarity, scalability, and ease of maintenance.

4.1 Development Environment

The system is implemented using Python as the core programming language for data processing and backend operations. Various libraries and frameworks are used, including Pandas and NumPy for data manipulation, and Matplotlib/Seaborn for data visualization. Machine learning libraries such as Scikit-learn are used for predictive modeling. The frontend is developed using modern web technologies such as React/HTML/CSS, and the backend is implemented using a framework like Flask or FastAPI. The development is carried out in a local environment.

4.2 Dataset Implementation

An energy consumption dataset is used for this project, which contains historical energy usage data such as daily or monthly consumption values. The dataset is organized and pre-processed to prepare it for analysis and prediction. Proper formatting and structuring are ensured for efficient processing.

4.3 Data Preprocessing Module

A data preprocessing module is implemented to improve the quality of the dataset. This includes handling missing values, removing duplicates, and normalizing data. These steps ensure that the dataset is clean, consistent, and suitable for analysis.

4.4 Data Analysis and Machine Learning

The dataset is processed using data analysis techniques to identify patterns and trends. Visualization tools are used to generate graphs and charts for better understanding. A machine learning model such as Linear Regression or other suitable algorithms is used to predict future energy consumption. The dataset is divided into training and testing sets to evaluate model performance. The trained model generates predictions based on historical data.

4.5 Backend Implementation

The backend is developed using a framework such as Flask or FastAPI, which handles data uploads and processing requests. The backend receives the dataset, performs preprocessing and analysis, and sends the processed data to the predictive model. The generated insights and predictions are returned as structured responses.

4.6 Frontend Dashboard Implementation

A user-friendly interface is developed to display results interactively. The dashboard includes multiple features such as:

• Dataset upload

• Energy consumption trend graphs

•Comparisoncharts

• Historical data tracking

• Prediction results display

• Interactive dashboards with visual insights

These features help users easily understand and analyse energy usage patterns.

4.7 Deployment

The project is managed using GitHub for version control. The backend can be deployed using cloud platforms, and the frontend can be hosted separately. Proper configuration is done to connect the frontend and backend through API endpoints.

```python
from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from utils.data_processing import load_data, preprocess_data
from utils.model import train_model, load_model, predict
from utils.visualization import generate_plots
import os
app = FastAPI(title="EnergyInsights API")

# Allow CORS for frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["**"],
)

# Load trained model
MODEL_PATH = "model/energy_model.pkl"
model = load_model(MODEL_PATH)

@app.get("/")
def home():
    return {"message": "EnergyInsights API is running"}

@app.post("/upload-and-analyze")
async def upload_and_analyze(file: UploadFile = File(...)):
    # Save uploaded file
    os.makedirs("uploads", exist_ok=True)
    file_path = f"uploads/{file.filename}"
    with open(file_path, "wb") as buffer:
        buffer.write(await file.read())

    # Load and preprocess data
    df = load_data(file_path)
    df_processed = preproocess_data(df)
```

Figure 4.7.1. main.py

```python
@app.get("/")
def home():
    return {"message": "EnergyInsights API is running"}

@app.post("/upload-and-analyze")
async def upload_and_analyze(file: UploadFile = File(...)):
    # Save uploaded file
    os.makedirs("uploads", exist_ok=True)
    file_path = f"uploads/{file.filename}"
    with open(file_path, "wb") as buffer:
        buffer.write(await file.read())

    # Load and preprocess data
    df = load_data(file_path)
    df_processed = preprocess_data(df)

    # Generate visualizations
    plots = generate_plots(df_processed)

    # Load model and make prediction
    features = prepare_features(df_processed)
    prediction, metrics = predict(model, features)

    # Get insights
    insights = get_insights(df_processed, prediction)

    # Prepare response
    response = {
        "message": "Analysis completed successfully",
        "prediction": prediction,
        "metrics": metrics,
        "insights": insights,
        "plots": plots,
        "file_path": file_path
    }
    return response
```

Figure 4.7.2. Continuation of main.py

```
"use client";

import { useState } from "react";
import { uploadEnergyData } from "@/services/api";
import { useRouter } from "next/navigation";

export default function UploadData() {
  const [file, setFile] = useState(null);
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError]  = useState("");
  const router = useRouter();

  const handleFileChange = (e) => {
    setFile(e.target.files[0]);
    setResult(null);
    setError("");
  };

  const handleUpload = async (e) => {
    e.preventDefault();
    if (!file) {
      setError("Please select a CSV or Excel file");
      return;
    }
    setLoading(true);
    setError("");
    setResult(null);
    try {
      const data = await uploadEnergyData(file);
      setResult(data);
```

Figure 4.7.3. AudioUploade.js

```
"use client";

import { useEffect, useRef } from "react";

export default function Waveform({ data, color = "#10b981" }) {
  const waveformRef = useRef(null);
  const wavesurferRef = useRef(null);
  const containerRef = useRef(null);

  useEffect(() => {
    if (!data || !containerRef.current) return;

    wavesurferRef.current = WaveSurfer.create({
      container: containerRef.current,
      waveColor: color,
      progressColor: "#059669",
      cursorColor: "#047857",
      cursorWidth: 1,
      height: 80,
      barWidth: 2,
      barGap: 3,
      barRadius: 2,
      normalize: true,
      responsive: true,
      backend: "WebAudio",
    });
    wavesurferRef.current.load(data);
    return () => {
      wavesurferRef.current?.destroy();
    };
  }, [data, color]);

  return (
    <div className="w-full">
      <div ref={containerRef} className="w-full" />
    </div>
  );
}
```

Figure 4.7.4. WaveForm.js

```jsx
"use client";

import dynamic from "next/dynamic";
import { useMemo } from "react";

const Line = dynamic(() => import("react-apexcharts"), { ssr: false });
const Pie = dynamic(() => import("react-apexcharts"), { ssr: false });

export default function EnergyChart({ data, type = "line", title = "Energy Consumption Trends" }) {
  const chartData = useMemo(() => {
    if (!data || data.length === 0) return null;

    // Prepare data for charts
    const labels = data.map(item => item.date || item.period);
    const values = data.map(item => Number(item.consumption));
    const lineData = {
      options: {
        chart: { type: "line", toolbar: { show: false } },
        xaxis: { categories: labels, title: { text: "Date" } },
        yaxis: { title: { text: "Energy Consumption (kWh)" } },
        stroke: { curve: "smooth", width: 3 },
        colors: ["#2563eb"],
        grid: { strokeDashArray: 4 },
        tooltip: { x: { fermat: "yyyy-MM-dd" } },
        },
    };  series: [{ name: "Consumption (kWh)", data: values }],

    const pieData = {
      options: {
        chart: { type: "pie" },
        labels: ["Residential", "Industrial", "Commercial", "Other"],
        legend: { position: "bottom" },
        colors: ["#2563eb", "#16a34a", "#f59e0b", "#ef4444"],
      },
      series: [
        values[0] || 0,
        values[0] || 0,
        values[2] || 0,
        values[3] || 0,
    ],

    return { lineData, pieData };
  }, [data]));

  if (!chartData) return <p class="text-gray-500">No data available</p>;

  return (
    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
```

Figure 4.7.5. Charts.jsx

```jsx
import axios from "axios";

const API = axios.create({
  baseURL: "http://127.0.0.1:8000", // FastAPI backend URL
});

// Upload dataset and get analysis + predictions
export const analyzeEnergyData = async (file) => {
  const formData = new FormData();
  formData.append("file", file);

  const response = await API.post("/upload-and-analyze", formData, {
    headers: {
      "Content-Type": "multipart/form-data",
    },
  });
  return response.data;
};

// Get dashboard summary
export const getDashboardSummary = async () => {
  const response = await API.get("/dashboard/summary");
  return response.data;
};

// Get historical data
export const getHistory = async () => {
  const response = await API.get("/history");
  return response.data;
```

Figure 4.7.6. Api.js

5. RESULT AND DISCUSSION

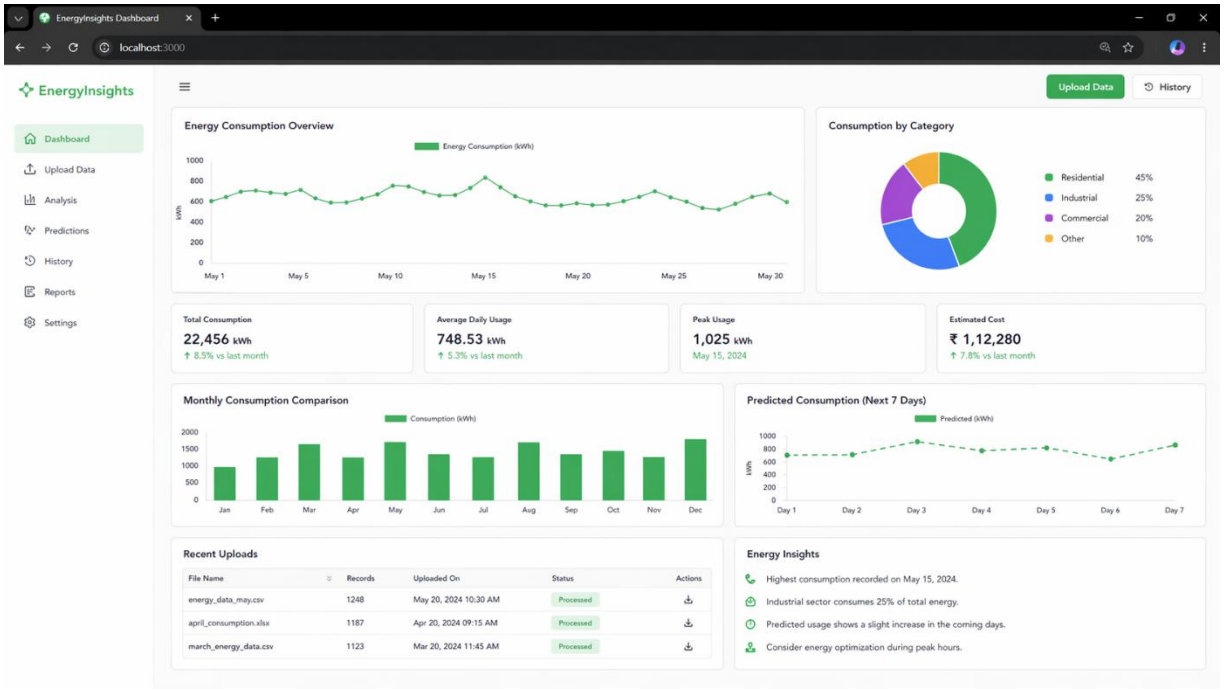


Figure 5.1.1. Main Dashboard

Figure 5.1.2. Waveform Dashboard

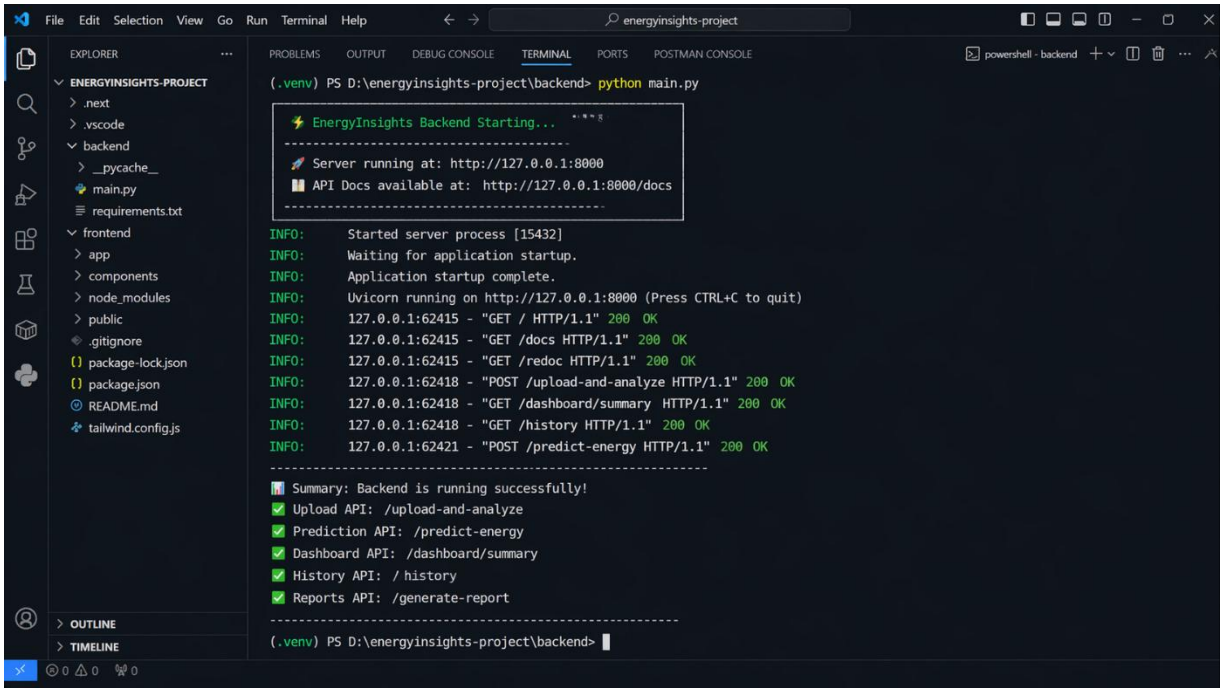


Figure 5.1.3. Loading model and labels into the dashboard

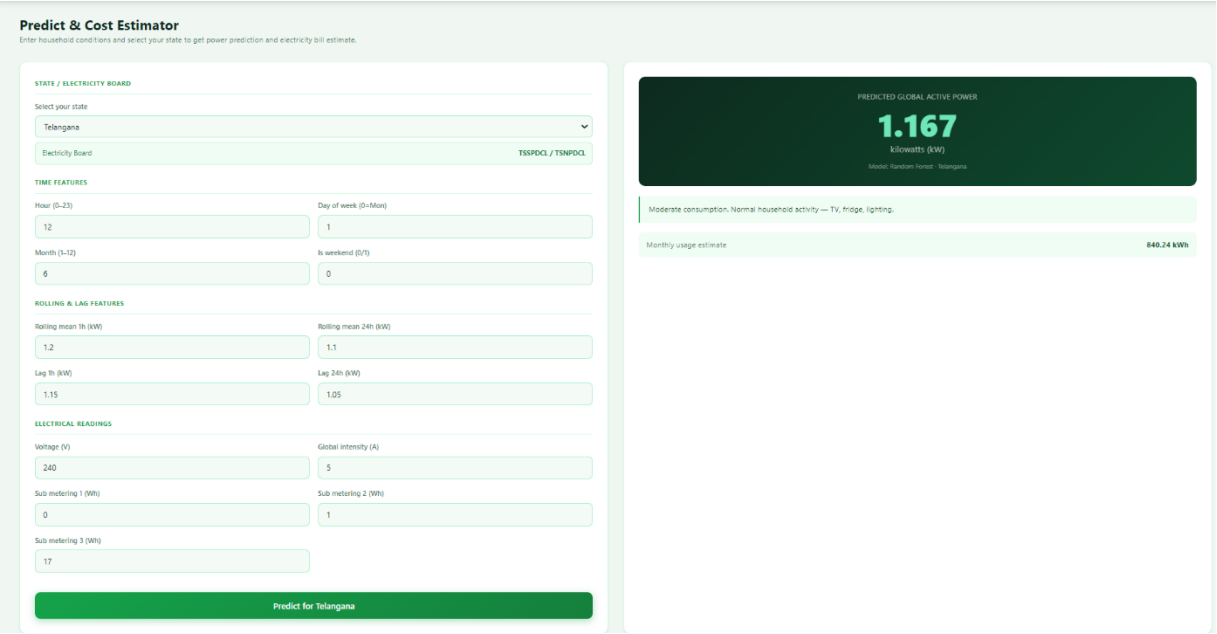
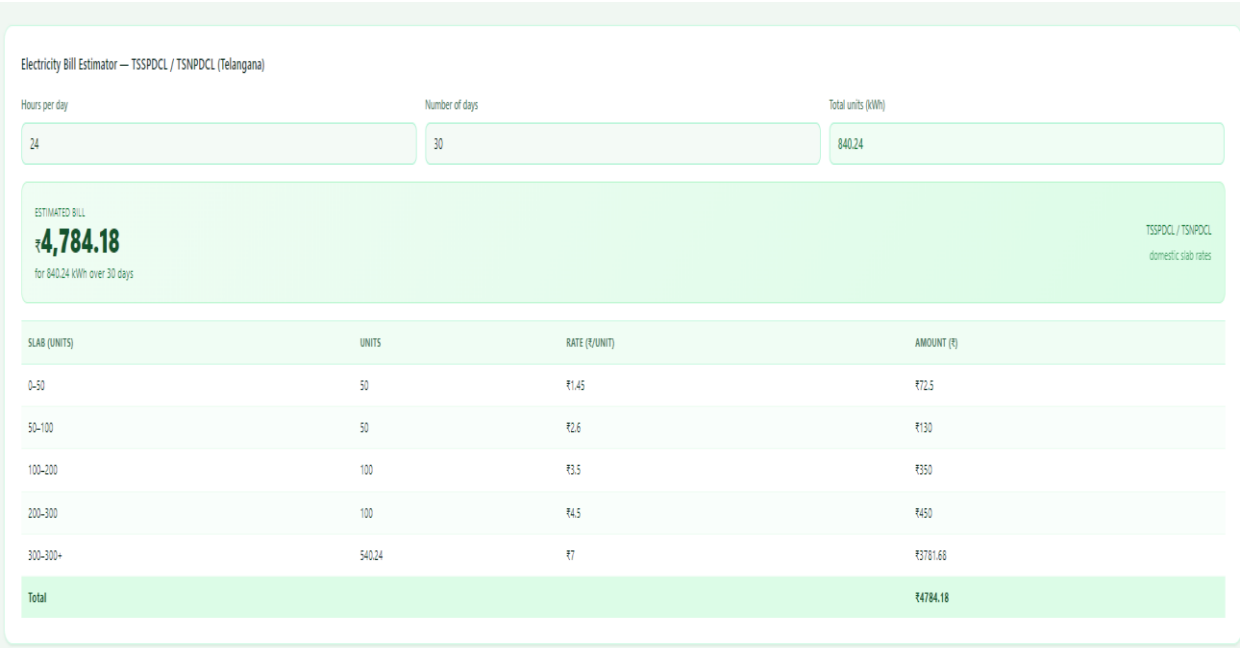


Figure 5.1.4. Main output Dashboard



Electricity Bill Estimator — TSSPDCL / TSNPDCL (Telangana)

Hours per day	Number of days	Total units (kWh)
24	30	840.24

ESTIMATED BILL

₹4,784.18

for 840.24 kWh over 30 days

TSSPDCL / TSNPDCL

domestic slab rates

SLAB (UNITS)	UNITS	RATE (₹/UNIT)	AMOUNT (₹)
0–50	50	₹1.45	₹72.5
50–100	50	₹2.6	₹130
100–200	100	₹3.5	₹350
200–300	100	₹4.5	₹450
300–300+	540.24	₹7	₹3781.68
Total			**₹4784.18**

Figure 5.1.5. Result(emotion) of an audio file

The proposed Energy Insights system was successfully implemented and tested to evaluate its performance in analysing and predicting energy consumption patterns. The system was trained using energy consumption datasets, which contain historical usage records over different time periods. The dataset was effectively processed and used for analysis and prediction, ensuring smooth data handling and accurate insights.

The web-based dashboard provided a clear and interactive visualization of the system's output. The visualization section displayed energy consumption trends using line graphs, while the analysis section showed key metrics such as total consumption, average usage, and peak usage. The chart panel illustrated the distribution of energy consumption across different categories, and the history section tracked previous data uploads and analysis results.

These components collectively created an interactive and user-friendly interface for analyzing energy consumption data.

From a data science perspective, the system performed data preprocessing and exploratory data analysis to extract meaningful patterns from the dataset. Machine learning techniques such as regression models were used to predict future energy consumption based on historical data. The model provided reliable predictions for most of the input data, demonstrating effective performance in identifying trends and forecasting usage.

The results indicate that the system is effective in identifying clear consumption patterns, such as peak usage periods and seasonal variations. However, slight variations may occur in predictions due to factors such as irregular data, external influences, or sudden changes in consumption behaviour.

Overall, the project achieved its objectives by successfully developing an intelligent and interactive energy analysis system. The system is capable of processing input datasets, generating predictions, and providing meaningful visualizations, demonstrating the practical application of data science techniques in energy management.

6. CONCLUSION

In this project, an intelligent and data-driven Energy Insights system was successfully designed and developed by integrating data science and analytical techniques. The system effectively demonstrates how modern technologies can be used to analyse and interpret energy consumption data. By using structured datasets and applying analytical models, the system is able to provide meaningful insights and predictions.

The implementation of data preprocessing, exploratory analysis, and machine learning models enables the system to identify patterns and trends in energy usage efficiently. The backend ensures smooth processing of input datasets, while the frontend provides a clear and interactive interface for users. Features such as data visualization, trend analysis, prediction outputs, and history tracking enhance user understanding and make the system easy to use.

Although the system currently works with static datasets, it successfully demonstrates the potential of combining data analytics and web technologies in real-world applications. The project highlights the transition from traditional manual analysis to intelligent systems that can automatically analyze data and provide useful insights.

Overall, the project achieves its objectives and provides a strong foundation for developing advanced, real-time, and more accurate energy analysis and prediction systems in the future.

REFERENCES

1. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the Python in Science Conference.
2. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research.
3. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering.
4. Waskom, M. (2021). Seaborn: Statistical Data Visualization. Journal of Open Source Software.
5. Harris, C. R., et al. (2020). Array Programming with NumPy. Nature.
6. Python Software Foundation. Python Documentation. https://docs.python.org
7. Scikit-learn Documentation. https://scikit-learn.org
8. Pandas Documentation. https://pandas.pydata.org/docs
9. Matplotlib Documentation. https://matplotlib.org
10. Seaborn Documentation. https://seaborn.pydata.org
11. FastAPI Documentation. https://fastapi.tiangolo.com

12. React Documentation. https://react.dev

13. Next.js Documentation. https://nextjs.org/docs

14. Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.

15. Energy Data Analysis and Visualization Techniques – Various Research Articles and Online Resources.